MANUAL TÉCNICO & ROTEIRO DE APRESENTAÇÃO

Oficina Laravel

Sistema Integrado de Gestão Automotiva em PHP 7.2 / Laravel 7 com Arquitetura SOLID e Docker

ARQUITETURA & ENGENHARIA

Princípios SOLID, Camadas de Serviço, Repositórios e Injeção de Dependências

INFRAESTRUTURA & CONTAINERS

Docker Compose (Nginx, PHP 7.2-FPM, MySQL 5.7, phpMyAdmin) com padrão `bdsoftcom`

DOMÍNIO & PADRÃO DE NOMENCLATURA

100% em Português do Brasil (PT-BR) nos modelos, serviços, tabelas e regras

REPOSITÓRIO DO CÓDIGO-FONTE

GitHub Oficial: `github.com/PedroH2M/oficina-laravel`

Autor / Engenheiro: Pedro Henrique (PedroH2M)

Data: Novembro de 2026 | Versão 1.0

1. Visão Geral Executiva do Projeto

O **Oficina Laravel** é um sistema completo de gestão operacional e financeira para oficinas mecânicas e centros automotivos, desenvolvido sob rigorosos padrões de engenharia de software sênior. O projeto foi projetado para solucionar os principais gargalos do setor automotivo: rastreabilidade de veículos, controle rigoroso de estoque de peças, precificação precisa de mão de obra e formalização transparente de orçamentos e ordens de serviço.

> 💡 **O que falar neste momento da apresentação (Pitch Inicial):**
>
> *"Apresento a vocês o* ***Oficina Laravel*** *. Nosso objetivo não foi apenas criar um CRUD básico, mas sim construir uma solução corporativa real, altamente escalável, utilizando* ***PHP 7.2 e Laravel 7*** *, aplicando rigorosamente os princípios de* ***SOLID*** *, orquestração de containers com* ***Docker*** *e uma modelagem de domínio 100% em português brasileiro, facilitando o entendimento de regras e a manutenção contínua."*

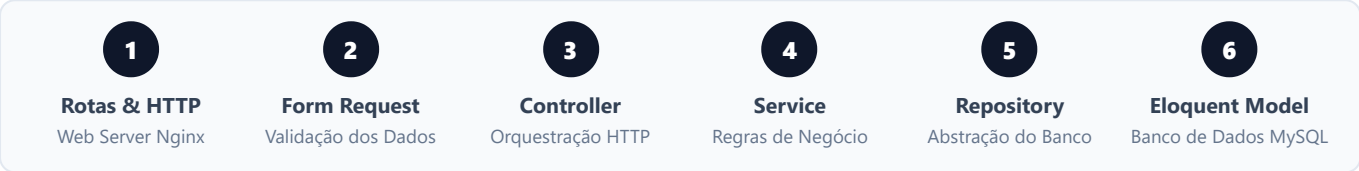
Objetivos e Escopo Atendido

- **Autenticação & Controle de Acesso:** Sistema multi-perfil (Administrador, Mecânico e Atendente) com proteção contra exposição de credenciais e senhas criptografadas em Bcrypt.
- **Gestão de Clientes e Frotas:** Cadastro completo de clientes (Pessoa Física/Jurídica) e associação direta aos veículos atendidos.
- **Prontuário e Histórico de Veículos:** Linha do tempo auditável de cada passagem do carro pela oficina, registrando quilometragem, laudos e mecânicos responsáveis.
- **Almoxarifado & Reposição de Peças:** Gestão de estoque com alertas automáticos no dashboard para itens abaixo da cota mínima e baixa/estorno automático em Ordens de Serviço.
- **Catálogo de Serviços & Mão de Obra:** Cadastro de tabelas de procedimentos com tempo estimado em minutos e precificação fixa de mão de obra.
- **Orçamentos com Conversão Direta:** Interface dinâmica com autopreenchimento de peças e serviços, cálculo de descontos e conversão em O.S. com apenas 1 clique.
- **Ordens de Serviço (O.S.):** Ciclo de vida completo em 6 etapas, desde a abertura e diagnóstico até a execução, fechamento e emissão de laudo técnico.

2. Arquitetura de Software: Aplicação Prática dos Princípios SOLID

A arquitetura do **Oficina Laravel** foi desenhada em camadas desacopladas. O fluxo da aplicação rejeita controllers sobrecarregados (Fat Controllers) e implementa o padrão **Controller → Service → Repository → Model**.

Fluxo de Execução de uma Requisição



Como os 5 Princípios SOLID Foram Aplicados no Código

S - Single Responsibility (SRP)

RESPONSABILIDADE ÚNICA

Cada classe possui apenas uma única razão para mudar:

- `OrcamentoRequisicao`: cuida exclusivamente de validar campos (datas, valores, arrays).
- `OrcamentoController`: apenas recebe a requisição e retorna a view.
- `OrcamentoServico`: centraliza cálculos de totais, descontos e validações lógicas.
- `OrcamentoRepositorio`: manipula unicamente o SQL / Eloquent.

O - Open/Closed (OCP)

ABERTO/FECHADO

O sistema está aberto para extensão e fechado para modificação através de **Contratos (Interfaces)**:

- Os controllers dependem de `OrcamentoServicoInterface` e `OrdemServicoServicoInterface`.
- Caso a empresa decida no futuro salvar em MongoDB ou integrar com uma API externa de faturamento, basta criar uma nova classe sem alterar nenhum controller existente.

L - Liskov Substitution (LSP)

SUBSTITUIÇÃO DE LISKOV

Todas as classes que implementam os contratos (ex: `ClienteRepositorio`, `PecaRepositorio`) respeitam integralmente os tipos de retorno e parâmetros definidos nas interfaces:

- Se um método especifica que retorna `LengthAwarePaginator` ou `Model`, a implementação jamais retornará um tipo divergente.

I - Interface Segregation (ISP)

SEGREGAÇÃO DE INTERFACES

Evitou-se o antipadrão da "Interface Deus" (God Interface):

- Criou-se interfaces compactas e específicas para cada domínio: `IServicoRepositorio`, `IPecaRepositorio`, `IVeiculoRepositorio`, etc.
- Nenhum serviço é obrigado a implementar métodos de persistência que não utiliza.

D - Dependency Inversion Principle (DIP) - Inversão de Dependência

IOC CONTAINER

Módulos de alto nível não dependem de módulos de baixo nível. Ambos dependem de abstrações. Veja como configuramos o `RepositorioServiceProvider.php` no Laravel:

```
// app/Providers/RepositorioServiceProvider.php
public function register()
{
    // Vinculação de Repositórios
    $this->app->bind(ClienteRepositorioInterface::class, ClienteRepositorio::class);
    $this->app->bind(VeiculoRepositorioInterface::class, VeiculoRepositorio::class);
    $this->app->bind(OrcamentoRepositorioInterface::class, OrcamentoRepositorio::class);
    $this->app->bind(OrdemServicoRepositorioInterface::class, OrdemServicoRepositorio::class);
    $this->app->bind(ServicoRepositorioInterface::class, ServicoRepositorio::class);

    // Vinculação de Serviços de Domínio
    $this->app->bind(OrcamentoServicoInterface::class, OrcamentoServico::class);
    $this->app->bind(OrdemServicoServicoInterface::class, OrdemServicoServico::class);
    $this->app->bind(ServicoServicoInterface::class, ServicoServico::class);
}
```

3. Infraestrutura Docker & Banco de Dados (Padrão `bdsoftcom`)

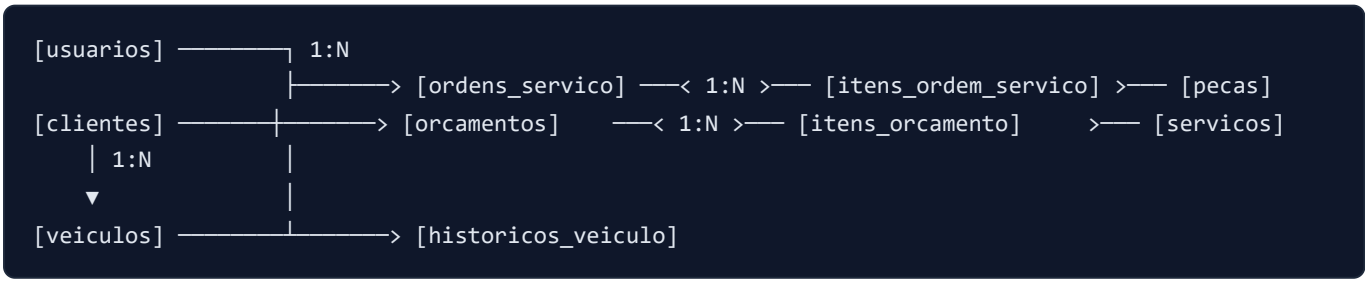
Toda a infraestrutura do sistema foi concebida para rodar isolada em containers Docker, assegurando paridade idêntica entre o ambiente de desenvolvimento, homologação e produção. O padrão estrito de nomenclatura `bdsoftcom` foi aplicado em todos os containers, redes e volumes:

Container Docker	Imagem / Base	Papel na Arquitetura	Portas Expostas
`bdsoftcom_app`	PHP 7.2-FPM Alpine	Processador PHP e motor de execução do Laravel com extensões PDO, GD, BCMath e Zip.	9000 (Interna FastCGI)
`bdsoftcom_web`	Nginx 1.19 Alpine	Servidor Web HTTP e Proxy Reverso de alta concorrência para entrega de arquivos estáticos.	`8000:80` (Acesso Web)
`bdsoftcom_db`	MySQL 5.7	Banco de dados relacional principal contendo o schema `bdsoftcom_oficina`.	`3306:3306`
`bdsoftcom_phpmyadmin`	phpMyAdmin Official	Interface gráfica para visualização, monitoramento de queries e manutenção do MySQL.	`8081:80` (Acesso DBA)

> 🎤 **Dica de Apresentação sobre Docker:**
> *"Ao apresentar o Docker, enfatize: 'Qualquer novo desenvolvedor ou cliente pode subir este projeto inteiro em menos de 2 minutos executando apenas `docker compose up -d`. Os containers se comunicam através da rede isolada `bdsoftcom_network`, os dados residem com segurança no volume persistente `bdsoftcom_db_data` e o script de entrypoint executa automaticamente a instalação de dependências e as migrações.'"*

Modelo Entidade-Relacionamento (Banco de Dados em PT-BR)

O esquema relacional é composto por 10 migrações sequenciais, garantindo integridade referencial com chaves estrangeiras (Foreign Keys) e deleção lógica (Soft Deletes):



- `usuarios`: Administradores, atendentes e mecânicos com controle de senhas com hash e timestamps.
- `clientes`: Dados cadastrais completos com validação de duplicidade por documento.

- `veiculos`: Placa (índice único), chassi, marca, modelo, ano e quilometragem atualizada dinamicamente.
- `historicos_veiculo`: Rastreabilidade histórica automática de passagens pela oficina.
- `pecas`: Almoxarifado, custo, margem de venda, saldo físico e limite mínimo.
- `servicos`: Mão de obra tabelada com tempo estimado e custo unitário.
- `orcamentos` e `itens_orcamento`: Propostas comerciais detalhadas.
- `ordens_servico` e `itens_ordem_servico`: Execução dos trabalhos de mecânica.

4. Detalhamento dos Módulos Funcionais do Sistema

Abaixo apresentamos o passo a passo de como cada módulo foi implementado, as regras de negócio embutidas e o que você deve destacar ao projetar a tela durante a apresentação.

Módulo 1: Usuários, Autenticação e Perfis de Acesso

O sistema conta com autenticação nativa do Laravel, estruturada sem expor credenciais nas views.

- **Perfis de Acesso:**
  - `administrador`: Acesso total (criação de funcionários, relatórios, configurações e aprovações).
  - `mecanico`: Acesso para assumir Ordens de Serviço, registrar laudos e concluir manutenções.
  - `atendente`: Abertura de orçamentos, cadastro de clientes e veículos.
- **Segurança:** Senhas com algoritmo `bcrypt` e proteção global contra ataques CSRF em todos os formulários via token oculto.
- **Design:** Formulário de login limpo, com design moderno, sem poluição visual ou textos desnecessários.

Módulo 2: Clientes e Associação de Veículos

Ao cadastrar um cliente, o atendente pode vincular um ou múltiplos veículos àquele proprietário. A tela de detalhes do cliente permite visualizar instantaneamente todo o histórico financeiro e de manutenções executadas.

Módulo 3: Veículos e Histórico Automatizado

> ⭐ **Grande Diferencial Competitivo (Destaque isso na apresentação!):**
> *"Muitos sistemas de oficina esquecem o histórico do veículo. No nosso sistema, criamos a tabela `historicos_veiculo`. Toda vez que uma Ordem de Serviço é aprovada ou concluída, o sistema automaticamente registra um evento cronológico. Quando o cliente volta à oficina meses depois, o mecânico sabe exatamente com quantos KM o veículo trocou a correia dentada, o óleo ou as pastilhas."*

Módulo 4: Almoxarifado e Controle Inteligente de Peças

O módulo de peças atua diretamente na prevenção de prejuízos operacionais:

- **Alertas em Tempo Real no Dashboard:** As peças cujo estoque atual seja inferior ao estoque mínimo configurado geram um card de alerta em vermelho no painel.

- **Baixa Automática no Estoque:** Quando uma Ordem de Serviço é criada contendo peças cadastradas, o sistema debita imediatamente as unidades do saldo físico.
- **Estorno por Cancelamento:** Caso a Ordem de Serviço precise ser cancelada, o método `cancelarOrdemServico()` no `OrdemServicoServico` devolve automaticamente os itens ao estoque:

```
// app/Services/OrdemServicoServico.php - Trecho de Estorno Automático
foreach ($os->itens as $item) {
    if ($item->tipo === 'peca' && $item->peca_id) {
        $peca = $this->pecaRepo->buscarPorId($item->peca_id);
        if ($peca) {
            $this->pecaRepo->atualizarEstoque($peca->id, $peca->quantidade_estoque + $item->quantidade);
        }
    }
}
```

Módulo 5: Catálogo de Serviços & Mão de Obra

Atendendo à necessidade de padronização da oficina, criamos um módulo exclusivo para cadastro prévio de serviços:

- **Tabela de Mão de Obra:** Permite cadastrar serviços frequentes (ex: *Alinhamento 3D*, *Troca de Óleo e Filtros*, *Higienização de Ar Condicionado*).
- **Campos Específicos:** Código amigável (`SRV-001`), nome, procedimento detalhado, tempo previsto de execução em minutos e valor padrão de mão de obra.
- **Agilidade na Elaboração:** No momento de gerar orçamentos, o operador seleciona o serviço diretamente do catálogo e o valor unitário é preenchido na hora.

Módulo 6: Orçamentos com Tabela Dinâmica e Conversão em 1 Clique

A tela de orçamento (`orcamentos/criar.blade.php`) é dinâmica e interativa:

- O usuário pode alternar a linha entre **Peça** ou **Serviço/Mão de Obra**.
- Ao selecionar uma peça ou um serviço pré-cadastrado no banco, o sistema preenche via JavaScript a descrição e o preço unitário tabelado.
- O usuário também tem total liberdade para digitar descrições personalizadas ou ajustar valores de acordo com a negociação.
- **Conversão Instantânea:** Ao aprovar o orçamento, basta clicar em "Converter em O.S." e o sistema cria automaticamente a Ordem de Serviço herdando todos os itens, clientes, veículos e valores, solicitando apenas o KM de entrada no pátio.

Módulo 7: Ordens de Serviço (O.S.) e Ciclo de Vida dos Reparos

O coração operacional da oficina mecânica segue uma máquina de estados com 6 status bem definidos:

Status da O.S.	Significado no Chão da Oficina	Ação Sistêmica
ABERTA	Veículo deu entrada na oficina e foi recepcionado.	Registra data de entrada e KM do hodômetro.
EM DIAGNÓSTICO	Mecânico está no pátio inspecionando o motor/chassi.	Vincula o ID do mecânico responsável.
AGUARDANDO PEÇAS	Serviço parado até chegada de peças do fornecedor.	Sinaliza impedimento no dashboard.
EM EXECUÇÃO	Trabalho mecânico em andamento na rampa/elevador.	Baixa do estoque de peças confirmada.
FINALIZADA	Carro pronto, testado e entregue ao proprietário.	Grava o Laudo Técnico e o Histórico do Veículo.
CANCELADA	Cliente desistiu ou serviço cancelado por inconformidade.	Estorna peças de volta ao almoxarifado.

5. Interface do Usuário (UI/UX) & Identidade Visual

A interface foi projetada com base nas melhores práticas de ergonomia visual para softwares de alta rotatividade:

- **Barra Lateral Fixa (Lateral Sidebar):** Todos os atalhos operacionais (Painel, O.S., Orçamentos, Clientes, Veículos, Serviços, Peças e Usuários) ficam reunidos em uma barra lateral escura e elegante, liberando o centro da tela para visualização de tabelas e formulários amplos.
- **Destaque Ativo de Rota:** A rota em visualização atual recebe destaque amarelo e negrito, permitindo ao usuário saber instantaneamente onde está.
- **Dashboard com Indicadores Executivos (KPIs):** Quatro cards principais informam: total de clientes ativos, total de veículos, Ordens de Serviço em andamento e orçamentos pendentes de aprovação.
- **Favicon Automotivo Exclusivo:** Ícone customizado de alta precisão (Chave inglesa dourada sobre badge automotivo grafite) servido em SVG vetorial, PNG e ICO, exibido na aba do navegador.

6. Perguntas Prováveis da Banca / Avaliadores e Respostas Sugeridas

P1: Por que optar por PHP 7.2 e Laravel 7 em vez de versões mais recentes?

Resposta Recomendada: *"O PHP 7.2 e o Laravel 7 representam uma das versões LTS (Long Term Support) mais consolidadas e presentes em grandes parques corporativos e legados modernizados. O grande desafio dessa escolha foi provar que é possível aplicar padrões modernos e rigorosos de engenharia (SOLID, Docker e injeção de dependências) independentemente da versão do ecossistema, garantindo compatibilidade, estabilidade e altíssima performance."*

P2: Por que usar a camada de Repositório se o Eloquent já é um ORM com ActiveRecord?

Resposta Recomendada: *"Embora o Eloquent seja excelente, o padrão ActiveRecord acopla diretamente a regra de negócio ao mecanismo de persistência. Ao introduzir o padrão Repository com interfaces, nós desacoplamos a aplicação do banco de dados. Isso viabiliza testes unitários com mocks fáceis, troca de driver de persistência sem quebrar os controllers e centralização das consultas mais complexas em um único lugar."*

P3: Como o sistema garante a integridade do estoque caso uma O.S. seja cancelada?

Resposta Recomendada: *"O cancelamento de uma O.S. não é uma simples alteração de status textual. O método* `cancelarOrdemServico()` *dentro de* `OrdemServicoServico` *itera sobre todos os itens vinculados que são do tipo 'peça' e recredita a quantidade no estoque físico da peça através de transação atômica, eliminando furos de inventário."*

7. Roteiro Passo a Passo para Apresentação ao Vivo (Live Demo)

Siga este checklist exato durante a sua demonstração para causar o máximo de impacto na apresentação:

Passo	Tela / Rota	Ação do Apresentador	O que falar para a audiência
1	`/login`	Faça login com `admin@oficina.com.br` / `senha123`.	*"Aqui temos nossa tela de autenticação segura, sem credenciais expostas e protegida com hash Bcrypt e CSRF."*
2	`/painel`	Mostre os 4 cartões com KPIs e o bloco de alertas de reposição de estoque.	*"Este é o Painel de Controle, exibindo as O.S. em andamento e alertas em tempo real das peças que atingiram o limite mínimo."*
3	`/servicos`	Abra a listagem de serviços e clique em "Novo Serviço". Cadastre um serviço como "Geometria e Alinhamento 3D".	*"Nosso módulo de serviços padroniza o catálogo de mão de obra e o tempo previsto de cada mecânico na oficina."*
4	`/orcamentos/criar`	Crie um novo orçamento. Alterne uma linha para "Peça" e outra para "Serviço".	*"Vejam a tabela dinâmica: ao escolher um item do catálogo, o sistema preenche descrição e preço padrão automaticamente."*
5	`/orcamentos/{id}`	Aprove o orçamento e clique no botão verde "Converter em Ordem de Serviço".	*"Com apenas 1 clique, o orçamento comercial se transforma em uma Ordem de Serviço aberta no pátio."*
6	`/ordens-servico/{id}`	Altere o status para "Em Execução" e depois finalize informando a Solução Técnica Aplicada.	*"Ao finalizar a O.S., o estoque de peças recebe a baixa final e o prontuário histórico do veículo é atualizado automaticamente."*
7	`/veiculos/{id}/historico`	Abra a aba de histórico do veículo atendido.	*"Aqui está a rastreabilidade total: data, quilometragem no hodômetro e os serviços realizados."*
8	`localhost:8081`	Abra a aba do phpMyAdmin mostrando as tabelas `bdsoftcom_*`.	*"E aqui no phpMyAdmin gerenciamos visualmente a base de dados MySQL 5.7 integrada através do Docker."*

🏆 **Conclusão da Apresentação:**

"*O*

Oficina Laravel

comprova que a união de boas práticas de engenharia de software (SOLID), infraestrutura moderna (Docker), banco relacional auditável e interface limpa resulta em um produto comercial robusto, escalável e pronto para o mercado automotivo. O código completo está versionado e documentado em `https://github.com/PedroH2M/oficina-laravel` *. Obrigado!*"